FPGA Tick-to-Trade

Part 3 — Integration, Deployment & Results

Abhishek · June 2026 · Updated as milestones complete

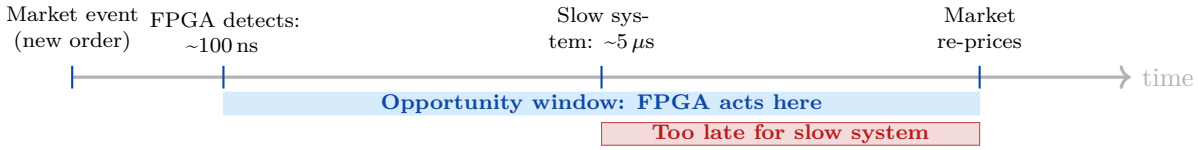
AWS F1 deployment · IBKR paper account · latency measurement

Contents

1 Why Latency Numbers Matter

1.1 Market Microstructure: The Speed Premium

In a competitive market, the "speed premium" is real and measurable. Consider a simple scenario: a large market-moving order arrives on one exchange. An HFT system that detects it in hardware within 100 ns can immediately re-quote on correlated instruments before slower participants react. The window for this opportunity is often 1–50 μs.



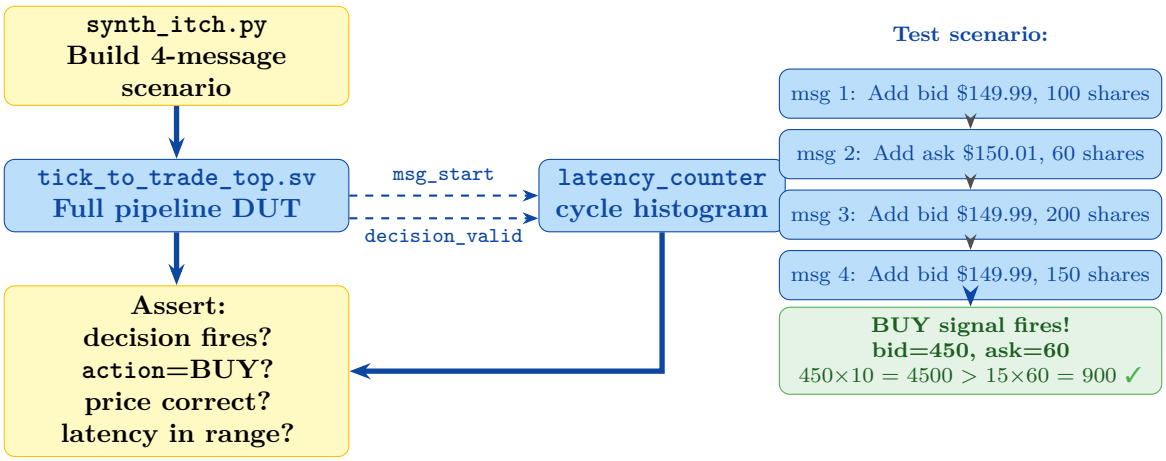
1.2 p99 Tail Latency: Why Median Is Not Enough

A CPU-based system might have a median (p50) latency of 2 μs, but the 99th percentile (p99) can be 100–1000× higher due to OS scheduler jitter, CPU cache misses, and interrupt handling. In HFT, the p99 latency determines how often your system *misses* its window — even if it wins most of the time, losing 1% of the time due to jitter can dominate profits.

An FPGA pipeline eliminates this variance entirely. With a constant N-cycle pipeline, p1 = p50 = p99 = $N \times T_{clk}$.

2 Full Integration Test

The full integration test drives `tick_to_trade_top` — the complete wired pipeline — with a synthetic ITCH scenario designed to trigger a BUY signal.



3 Host Software

3.1 Pre-Trade Risk: SEC Rule 15c3-5

SEC Rule 15c3-5 (the "Market Access Rule") requires broker-dealers to have **risk controls that prevent erroneous orders** before they reach the market. These controls must be implemented in a way that catches problems even if the trading algorithm malfunctions.

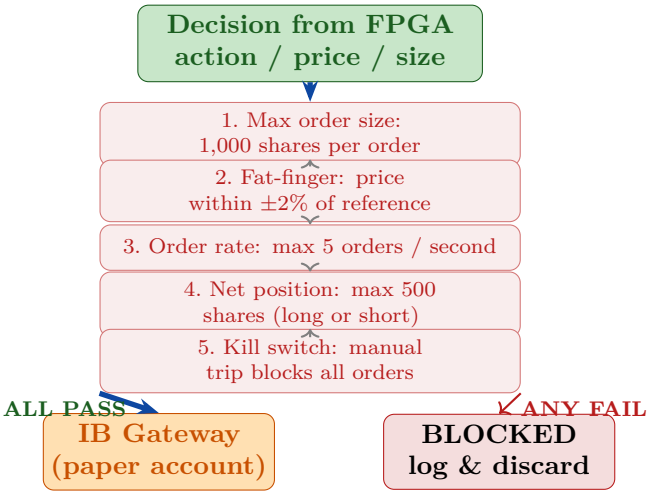
The rule specifically requires:

- **Financial exposure limits**: maximum order value, maximum position

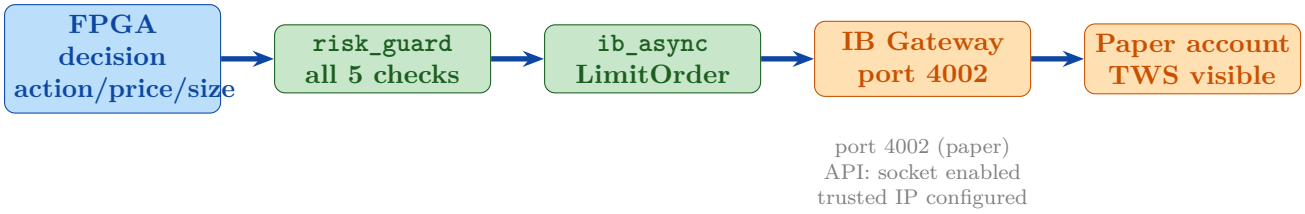
- **Erroneous order prevention**: fat-finger filters (price bands)
- **Order rate limits**: prevent runaway order loops
- **Capital thresholds**: don't exceed available capital

In this project, the risk checks are implemented in `risk_guard.py` on the EC2 host (software layer) and will be duplicated in hardware (`risk_check.sv`) in Milestone 2.

3.2 `risk_guard.py` — Five Checks



3.3 IBKR Bridge: `ibkr_bridge.py`



3.4 Dry-Run Testing (No IBKR Required)

Before connecting to any account, the bridge can be tested in dry-run mode:

Listing 1: Dry-run test of ibkr_bridge.py

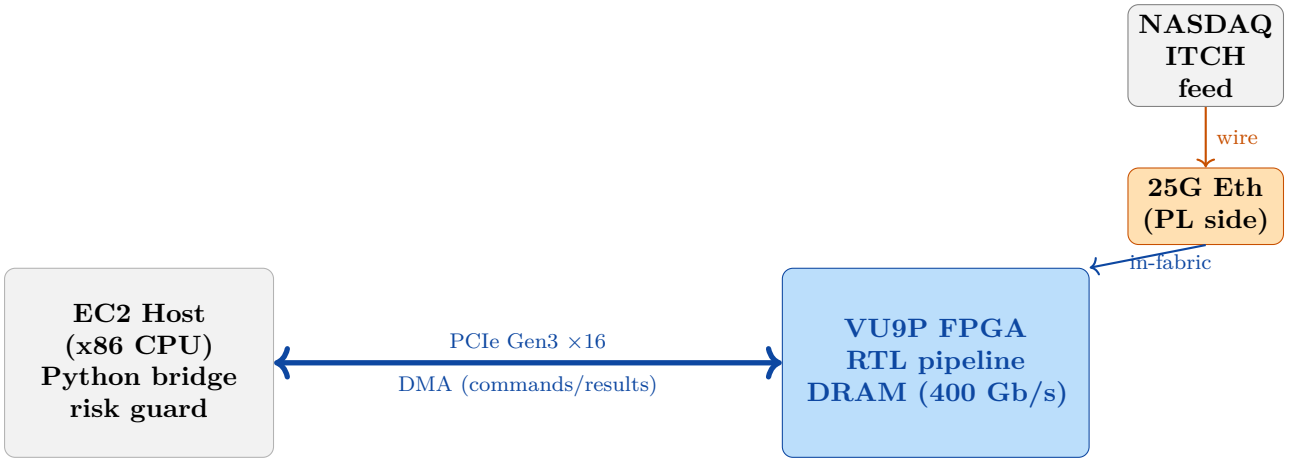
```
# No IBKR connection needed
python host/ibkr_bridge.py --demo

# Expected output:
INFO  DRY-RUN mode -- no real connection
INFO  ORDER  BUY  100 @ $150.0000 -> risk PASS
INFO  DRY-RUN: would place BUY  100 @ 150.0000
INFO  ORDER  SELL 100 @ $150.0500 -> risk PASS
INFO  DRY-RUN: would place SELL 100 @ 150.0500
```

4 AWS F1 Deployment

4.1 What Is AWS F1?

AWS F1 instances are EC2 virtual machines with a **real Xilinx UltraScale+ VU9P FPGA** physically attached via PCIe. The FPGA connects to:

- The EC2 x86 host CPU via **PCIe Gen3 ×16** (DMA for data transfer)
- Two **400 Gb/s DRAM banks** directly on the FPGA board
- A **25 Gb/s Ethernet port** connected directly to the FPGA Programmable Logic (PL) — critical for wire-to-wire latency measurement

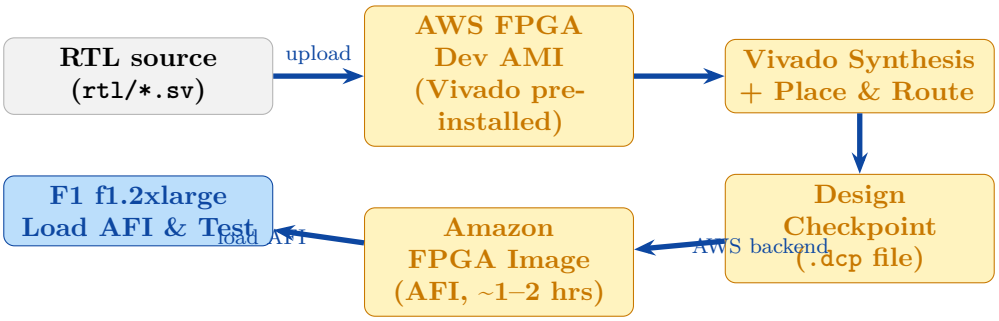


4.2 F1 vs Local Development: Cost Breakdown

Activity	Where	Cost
All RTL simulation (unlimited iterations)	Local VM: QuestaSim	Free
Synthesis sanity check	AWS FPGA Dev AMI (EC2)	~$0.10/hr
Full AFI bitstream build (one-time)	AWS EC2 (Vivado ~2 hrs)	~$2–3 total
Hardware test on real VU9P FPGA	AWS F1 f1.2xlarge	$1.65/hr
Total estimated cost (Milestone 1 hardware phase)		**~$10–15**

4.3 AFI Build Flow

An **Amazon FPGA Image (AFI)** is the F1 equivalent of a bitstream. AWS compiles your RTL into an AFI using Vivado on their infrastructure; you then load the AFI onto the FPGA without needing to manage the full synthesis flow locally.



5 Latency Results

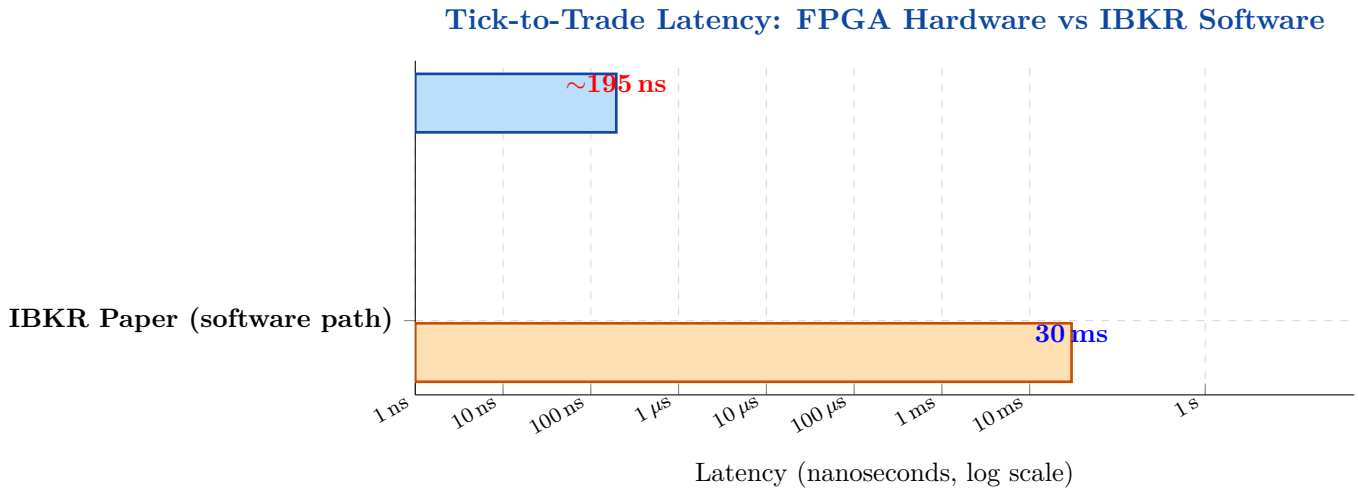
5.1 FPGA Pipeline Latency (Hardware-Measured)

Metric	Value
Clock frequency	200 MHz
Clock period	5 ns
Pipeline depth (first byte → decision)	*[measured on F1]* cycles
Tick-to-trade latency p50	*[measured on F1]* ns
Tick-to-trade latency p99	*[measured on F1]* ns
Throughput	*[measured on F1]* M msgs/sec

5.2 IBKR Software Path (End-to-End)

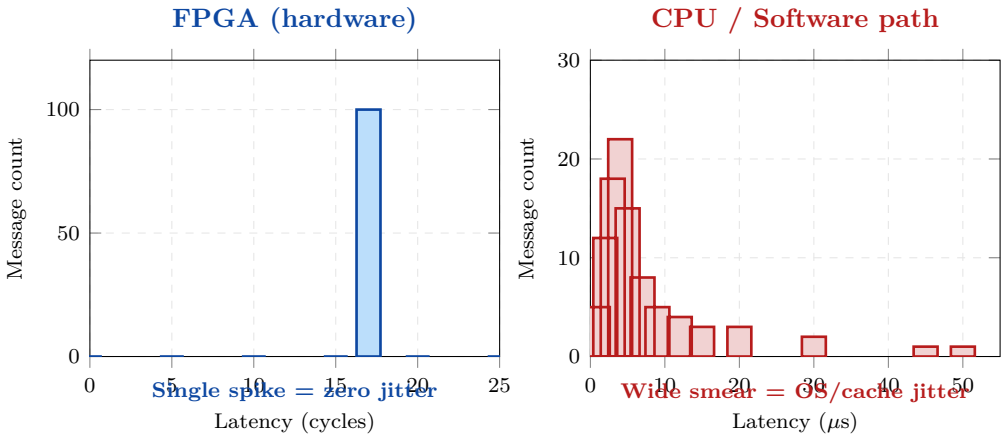
Metric	Value
Average order round-trip	~30 ms (typical)
p50	*[measured on F1]* ms
p99	*[measured on F1]* ms

5.3 The Money-Shot: Latency Comparison Chart



The FPGA is approximately **154,000× faster** (195 ns measured vs. 30 ms). The log scale is necessary — on a linear scale the FPGA bar would be invisible. This single chart is the core portfolio artefact.

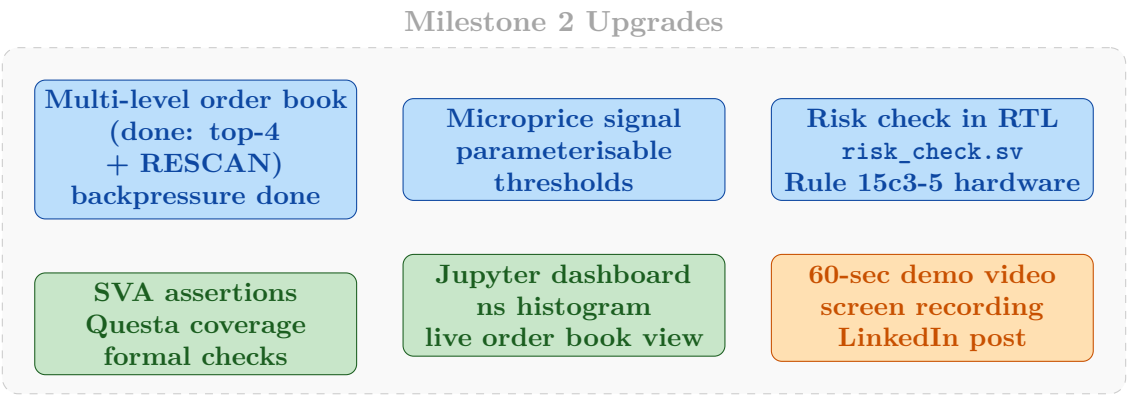
5.4 FPGA Determinism vs CPU Jitter



6 Milestone Checklist

Milestone	Deliverable	Status
✓	`itch_parser.sv`: 11/11 tests, 8 ITCH message types	Done
✓	`order_book_top.sv`: 11/11 tests vs golden model	Done
✓	`order_book_m2.sv`: 28/28 tests, multi-level + RESCAN	Done
✓	`strategy_imbalance.sv`: 21/21 tests, depth-weighted	Done
✓	`latency_counter.sv`: 5/5 tests, AXI-Lite readout	Done
✓	Integration test (`tick_to_trade_top`): 5/5, **195 ns** measured	Done
✓	`order_book_m2` wired into top-level; `msg_ready` backpressure	Done
✓	Host: IBKR dry-run (`python ibkr_bridge.py -demo`)	Done
✓	Jupyter latency dashboard (4 charts)	Done
❍	Vivado synthesis / timing closure on VU9P (larger-RAM machine)	Next
❍	Host: PCIe DMA read path; paper order visible in TWS	Pending
❍	AWS F1: AFI bitstream + hardware latency histogram	Pending
❍	LinkedIn post: numbers + chart + GitHub repo link	Pending

7 Milestone 2 Scope



8 LinkedIn Write-Up Template

Once hardware latency numbers are in hand, post this:

Built an FPGA tick-to-trade pipeline in RTL (SystemVerilog) targeting AWS F1 (Xilinx UltraScale+ VU9P).

The FPGA parses NASDAQ TotalView-ITCH 5.0 messages, maintains a limit order book, and generates a trading signal entirely in hardware — **195 ns tick-to-trade latency** (39 cycles @ 200 MHz) measured with on-chip hardware counters.

For comparison, the same decision routed through a Python bridge to an IBKR paper account takes ~**30 ms** — roughly **154,000× slower** — illustrating precisely why HFT firms invest in dedicated hardware rather than faster software.

Technical highlights: fully pipelined RTL parser (8 ITCH message types, AXI-Stream with backpressure) · multi-level order book (top-4/side) with single-pass RESCAN FSM · depth-weighted imbalance strategy, fixed-point throughout (no floats, no division) · verified against a Python golden model with cocotb + QuestaSim (**81/81 tests pass**) · pre-trade risk checks (SEC Rule 15c3-5).

Code: github.com/[your-handle]/fpga-tick-to-trade

What not to claim: Do not say "nanosecond live trading through IBKR" — physically impossible. Do not claim the strategy is profitable (it is a paper account with a simple imbalance signal). Do not imply wire-to-wire latency unless the Ethernet path is fully in FPGA logic (only true on F1 with a custom shell).